

DESIGN DOCUMENT

Titan SAP Data Mapping Module

PPC CBE Tracking Application

Scope: SAP-to-application data integration workflow

Prepared by: Athithya

Reviewer: Kauverysree

Document version: 1.0

Status: Draft — pending confirmation on items listed in Section 19

Contents

1	Module Objective
2	SAP Data Source Understanding
3	SAP Data Dictionary
4	Application Data Model (Reference)
5	SAP-to-Application Mapping Matrix
6	End-to-End Workflow
7	Validation Rules
8	Detailed Data Processing Flow
9	Field-Level Mapping Flow
10	Error Handling
11	Duplicate and Update Handling
12	Database Impact
13	API / Integration Contract (Proposed)
14	Security
15	Logging & Monitoring
16	Reconciliation
17	Synchronization Strategy
18	Business Rules
19	Open Questions / Decisions Required (Consolidated)
20	Final End-to-End Summary

This document defines the SAP-to-application integration workflow for the PPC CBE Tracking Application. Several sections depend on information not yet confirmed by the SAP team; these are highlighted throughout as **[OPEN]** rather than assumed. Section 19 consolidates the full list.

1. Module Objective

This module defines how data from Titan SAP is received, mapped, validated, transformed, and integrated into the PPC CBE Tracking Application. It sits at the boundary between SAP (the system of record for dispatch, purchase orders, vendor master, and stock movements) and the application's own database, which drives the sub-con material tracking workflow (M3 through M14).

The integration style is confirmed as **REST API**. The exact SAP endpoints, authentication mechanism, and full field-level payload schema are **[OPEN]** — see Section 19 for the consolidated list.

2. SAP Data Source Understanding

Based on the data you've requested from the SAP team, here is what's being pulled and why:

#	SAP Data	Why Needed	Where It's Used in This App
1	Delivery Challan (DC)	Verify the vendor delivery	M5 DC Verification, M7 Deviation Management
2	Purchase Order (PO)	Validate the inward material	M5 DC Verification (cross-check against DC)
3	Vendor Master	Validate the supplier	M4 Receiving, M15 Masters
4	Movement Type 101	Goods Receipt	M9 SAP Processing (UD Post confirmation)
5	Movement Type 313	Stock Transfer	M9 SAP Processing, M12 ZQMTL1 Processing
6	Movement Type 321	Quality to Unrestricted Stock	M11 QA Inspection
7	Material Document Number	Store the SAP reference after posting	M9 SAP Processing, M14 Reports (traceability)
8	Inventory / Stock Movements	Material movement history	M14 Reports, M13 Storage Management
9	Open PO / Pending Quantity	Pending receipt information	M2 Dashboard (pending counts), M5 DC Verification

This list tells us **what** categories of data are needed and **why**, but not yet the individual field names, data types, or exact payload shape within each category (for example, what fields make up "Vendor Master" as SAP will return it). That level of detail is **[OPEN — pending SAP team's field-level specification]**.

Movement types 101/313/321 already correspond to specific stages in the confirmed process flow: 101 is Goods Receipt at Gate/Sub-con inward, 313 is the stock transfer step described after UD Post (both in the original inward flow and again after ZQMTL1), and 321 is the Quality Inspection movement referenced when material clears QA. This gives us a mapping between SAP's internal movement vocabulary and the physical stages already documented in the architecture, which is useful context even before the exact field payloads are confirmed.

3. SAP Data Dictionary

The table below is a first-pass data dictionary built from the categories requested and the fields already referenced elsewhere in the architecture document (Section 5.3 ERD, M3 SAP Outward table). Fields marked **[OPEN]** need the SAP team to confirm exact name, type, and format.

SAP Data Category	Field (expected)	Data Type	Description	Status
Delivery Challan	DC Number	String	Unique challan identifier	Confirmed (referenced throughout process)
Delivery Challan	Line Item Count	Integer	Number of line items (max 13 per DC, per business rule)	Confirmed
Delivery Challan	Vendor Code	String	Links to Vendor Master	[OPEN] exact field name
Delivery Challan	Material Code	String	Links to Material Master	[OPEN] exact field name
Delivery Challan	Quantity (Front/ Back Case)	Numeric	Expected quantity per line item	Confirmed (Front Case / Back Case distinction exists in process)
Purchase Order	PO Number	String	Purchase order reference	[OPEN]
Purchase Order	PO Line Item	String/Integer	Line-level reference	[OPEN]
Purchase Order	Open Quantity	Numeric	Pending/undelivered quantity	[OPEN]
Vendor Master	Vendor Code	String	Unique vendor identifier	[OPEN]
Vendor Master	Vendor Name	String	Display name	Confirmed (used in M3 table)
Movement Type	Movement Type Code	String (101/313/321)	SAP goods movement code	Confirmed
Movement Type	Material Document Number	String	Reference number generated on posting	Confirmed (referenced in process as SAP reference)
Movement Type	Posting Date	Date	When the movement was recorded in SAP	[OPEN]
Stock Movement	Storage Location	String	SAP bin/location reference	[OPEN] — relationship to app's own Rack/Bin (M13) needs clarification
Stock Movement	Quantity Moved	Numeric	Quantity for that movement	[OPEN]
Open PO	Pending Quantity	Numeric	PO quantity not yet received	[OPEN]

This table should be treated as a draft skeleton, not a final contract. It needs to be validated line-by-line against whatever the SAP team actually returns once the REST endpoints are shared.

4. Application Data Model (Reference)

The application-side entities this SAP data needs to map into already exist in the architecture's ERD (Section 5.3): `VENDOR`, `DISPATCH`, `DELIVERY_CHALLAN`, `WHITE_BOX`, `DEVIATION`, `STORAGE_LOCATION`, and `SCAN_EVENT`. Section 12 (Database Impact) below maps each SAP category onto these specific tables and flags where new columns are needed.

5. SAP-to-Application Mapping Matrix

SAP Field	Application Entity	Application Field	Notes
DC Number	<code>DELIVERY_CHALLAN</code>	<code>dc_no</code>	Direct map
Line Item Count	<code>DELIVERY_CHALLAN</code>	<code>line_item_count</code>	Used to enforce the 13-line-item split rule
Vendor Code	<code>VENDOR</code>	<code>code</code>	Direct map, pending exact SAP field name
Vendor Name	<code>VENDOR</code>	<code>name</code>	Direct map
Material Code	<code>DISPATCH</code>	<code>material_code</code>	Direct map
PO Number	<code>DISPATCH</code> (<i>new field needed</i>)	<code>po_number</code>	Not currently in ERD — needs to be added
Movement Type + Material Document Number	<code>DISPATCH</code> / new <code>SAP_MOVEMENT</code> table	<code>sap_document_no</code> , <code>movement_type</code> , <code>posting_date</code>	Recommend a dedicated table rather than overloading <code>DISPATCH</code> — see Section 12
Open PO / Pending Quantity	<code>DISPATCH</code> (<i>derived, not stored</i>)	—	Likely computed from PO minus received-to-date rather than stored directly — [OPEN] , confirm with SAP team whether this is pushed as a value or must be calculated app-side
Stock Movement / Storage Location	<code>STORAGE_LOCATION</code>	<code>rack</code> , <code>bin</code>	[OPEN] — need to confirm whether SAP's storage location concept maps 1:1 to the app's Rack/Bin model or is a separate reference that needs reconciliation

Several rows above depend on fields still marked **[OPEN]** in Section 3. This matrix will need a second pass once those are confirmed.

6. End-to-End Workflow

Figure 1 — High-Level Integration Flow

The diagram below shows the full path data takes from Titan SAP into application usage.

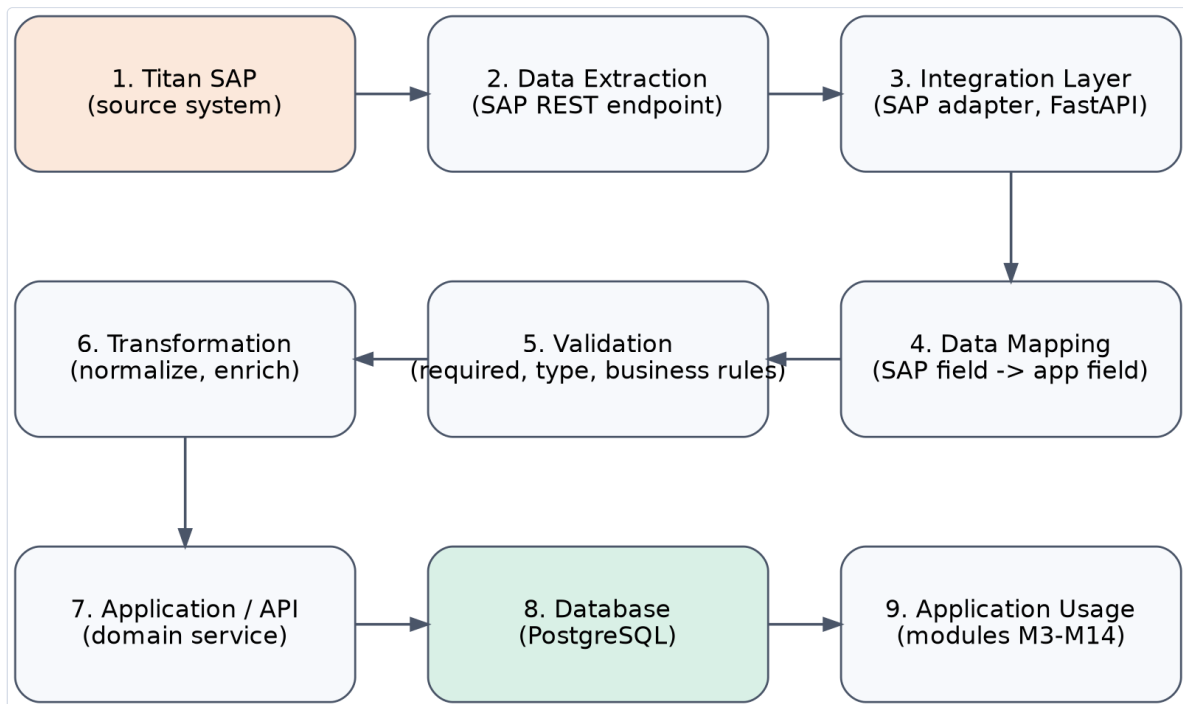


Figure 1 — High-level integration flow: Titan SAP through to application usage.

Stage 1 — SAP Data Availability. Titan SAP holds Delivery Challans, Purchase Orders, Vendor Master, and stock movement records (101/313/321) as they're created or updated within SAP's own processes — outside this application's control.

Stage 2 — Data Extraction. The confirmed integration style is REST, so extraction happens via the application calling SAP's REST endpoints (pull model) rather than SAP pushing to us — **[OPEN: confirm this is pull, not push/webhook]**. See Section 8 for the synchronization strategy options.

Stage 3 — Integration. The SAP adapter (already scaffolded as `SAPClientInterface` / `SAPRestClient` in the backend, per the architecture doc Section 5.2) receives the raw SAP response and passes it into the integration layer.

Stage 4 — Mapping. Each SAP field is mapped to its corresponding application entity/field per the matrix in Section 5.

Stage 5 — Validation. Every mapped record passes through the validation rules in Section 7 before being accepted.

Stage 6 — Transformation. Valid records are normalized (e.g., date formats, casing, unit conversions if any) and enriched (e.g., resolving a vendor code into the full vendor record already stored locally).

Stage 7 — Application Processing. The transformed record enters the relevant module's business logic (e.g., a DC record feeds M5 DC Verification).

Stage 8 — Database Persistence. The record is written to PostgreSQL, following the duplicate/update logic in Section 10.

Stage 9 — Application Usage. Once persisted, the data is available to the dashboard (M2), reports (M14), verification screens (M5/M6), and deviation handling (M7) as described in Section 2's usage column.

7. Validation Rules

Validation Type	Applies To	Rule
Required fields	All incoming records	DC Number, Vendor Code, Material Code, Quantity must be present
Data type	All fields	Must match expected type (string/numeric/date) before mapping proceeds
Format validation	DC Number, PO Number, Material Document Number	Must match expected SAP numbering format — [OPEN] , exact format/pattern not yet confirmed
Duplicate checking	DC Number + line item	See Section 10 for the full duplicate/update logic
Reference / master-data validation	Vendor Code, Material Code	Must resolve to an existing VENDOR / material master record in the application; unresolvable references are rejected, not auto-created — [OPEN: confirm this is the desired behavior, vs. auto-creating a stub record]
Business-rule validation	Line Item Count	Must not exceed 13 per DC (existing confirmed rule); if SAP sends more, the application's existing split logic applies
Identifier validation	Material Document Number	Must be unique per movement type + posting — exact uniqueness scope [OPEN]

8. Detailed Data Processing Flow

Figure 2 — Record-Level Processing with Decision Points

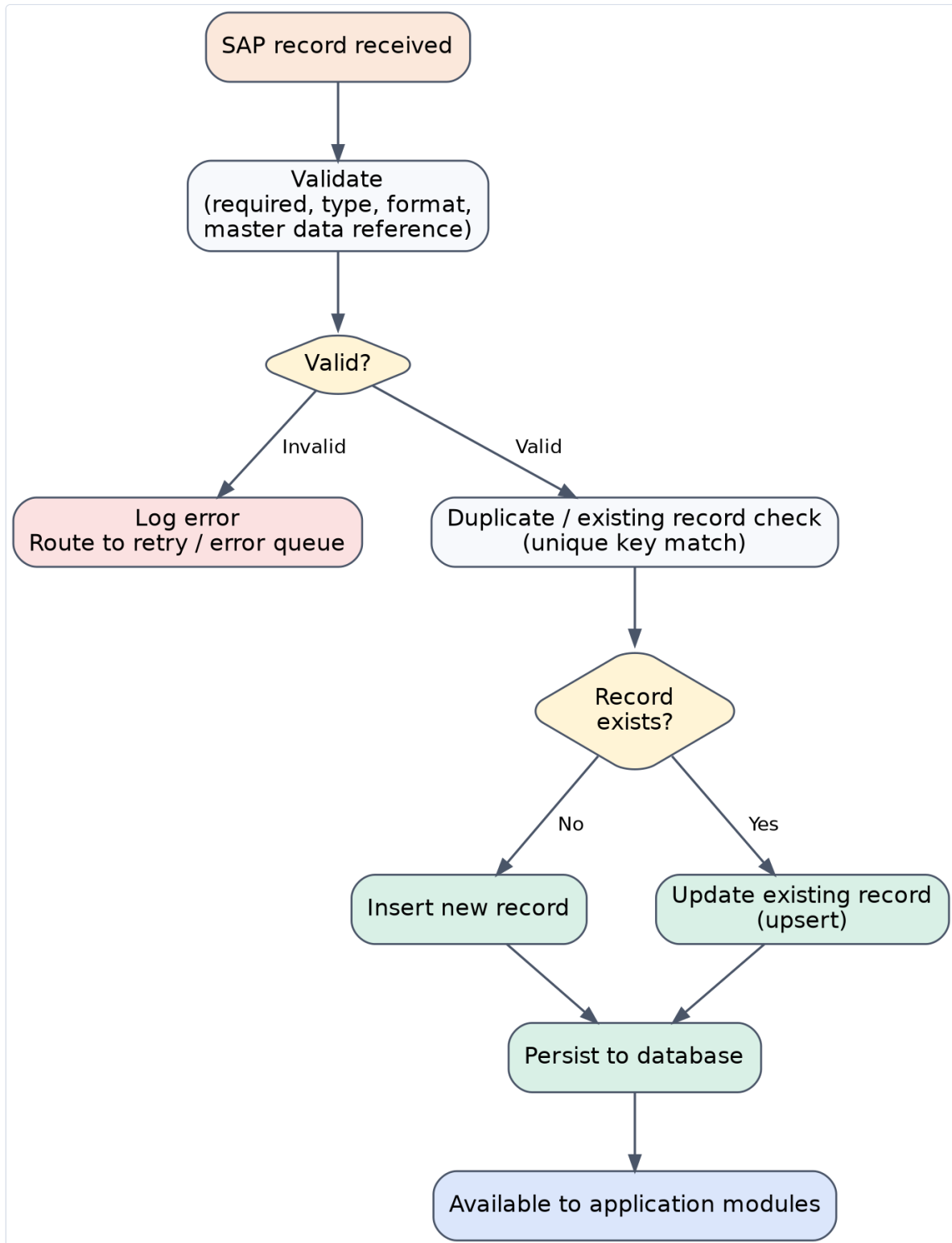


Figure 2 — Record-level processing flow with validation and duplicate/update decision points.

This shows what happens to a single incoming SAP record: validation, duplicate/existing check, and the resulting insert vs. update path, ending in persistence and availability to the rest of the application.

9. Field-Level Mapping Flow

Figure 3 — Per-Field Processing Chain

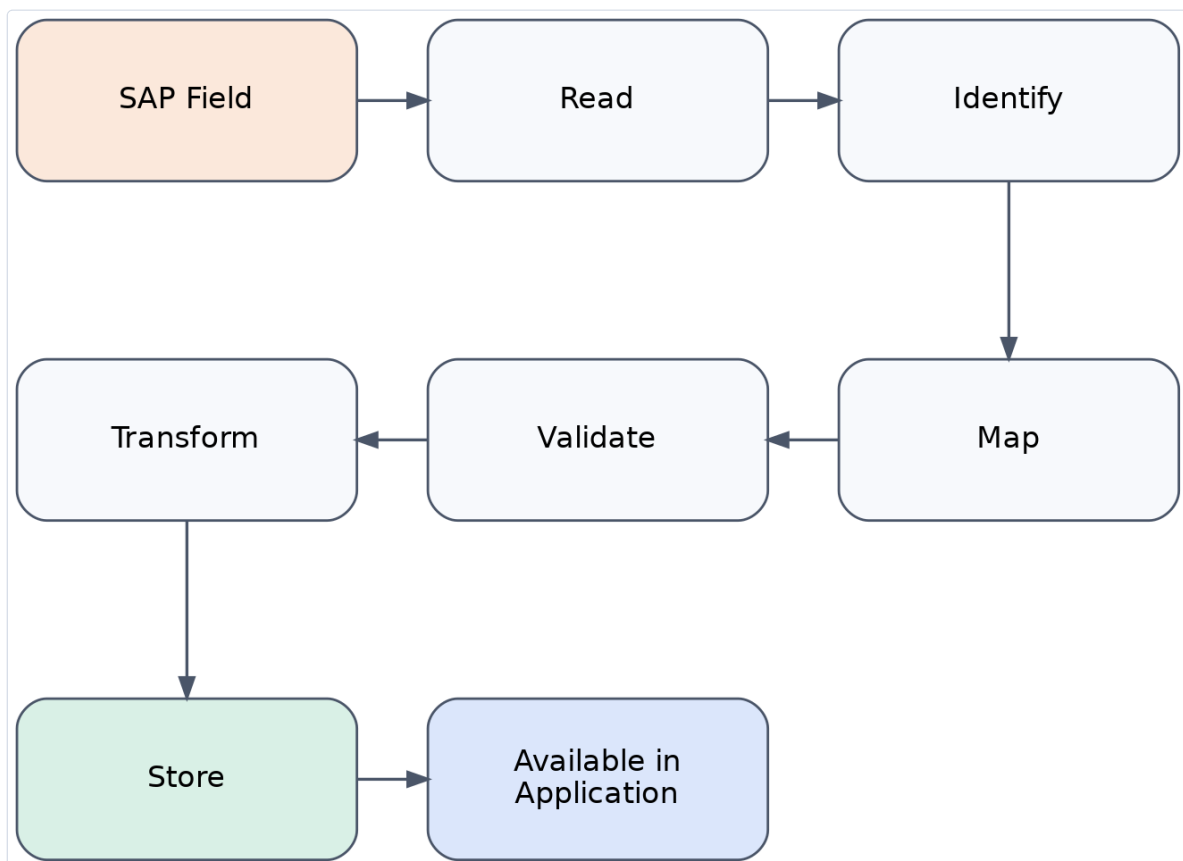


Figure 3 — Per-field processing chain, from raw SAP value to application availability.

Every individual SAP field goes through the same eight-step chain: **Read** the raw value from the SAP payload, **Identify** which application field it corresponds to (per Section 5's matrix), **Map** it into the target schema, **Validate** it against the rules in Section 7, **Transform** it into the application's expected format, **Store** it in the database, and make it **Available in Application**.

Worked example using a confirmed field:

DC Number → Read ("DC-0001" from the SAP payload) → Identify (maps to `DELIVERY_CHALLAN.dc_no`) → Map (assign to the DC record being built) → Validate (non-empty, format check) → Transform (no transformation needed, direct string) → Store (written to `delivery_challan` table) → Available in Application (surfaces in M5 DC Verification screen).

A worked example for a field still marked **[OPEN]** (e.g., Storage Location) can't be completed accurately until the SAP field's exact structure is confirmed.

10. Error Handling

Figure 4 — Error and Retry Flow

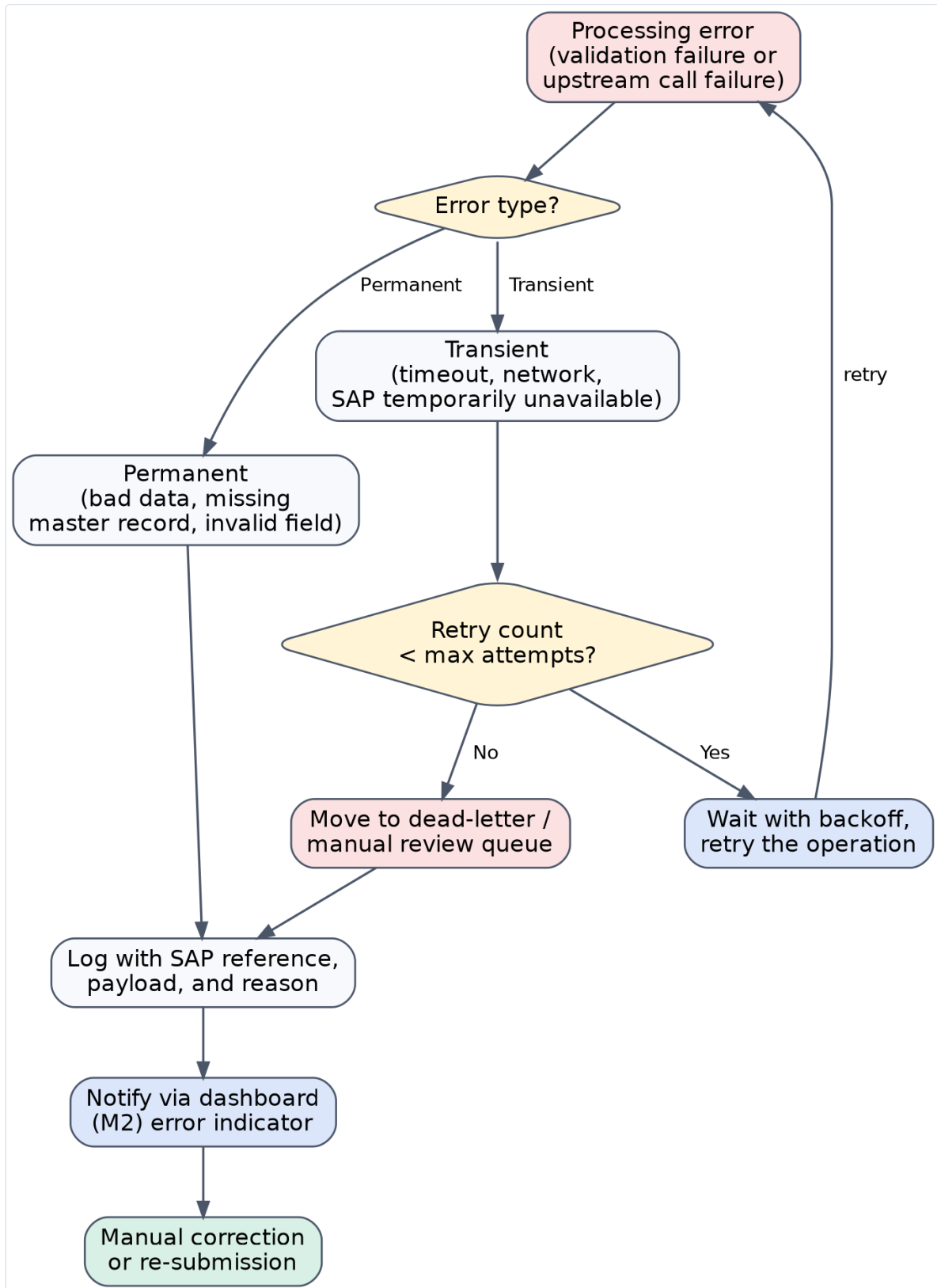


Figure 4 — Error classification and retry/dead-letter flow.

Errors are split into two categories:

Transient errors (timeouts, network failures, SAP temporarily unreachable) are retried automatically with backoff, up to a maximum attempt count. This fits the architecture's existing SAP adapter pattern (Section 5.2), which already anticipates retry/backoff on SAP calls.

Permanent errors (malformed data, an unresolvable vendor/material reference, a validation failure that isn't going to fix itself on retry) are logged immediately with the SAP reference and full payload, and routed to a manual-review state rather than retried indefinitely.

Both paths converge on structured logging (SAP reference, payload, reason) and a dashboard-visible indicator so failures aren't silent. **[OPEN]**: exact maximum retry count and backoff interval need a decision — proposed starting point is 3 attempts with exponential backoff (e.g., 30s, 2min, 10min), but this should be confirmed against how time-sensitive SAP data actually is in practice.

11. Duplicate and Update Handling

This is one of the areas most dependent on information not yet available. Rather than inventing rules, here is what can be proposed versus what genuinely needs a decision:

Proposed unique key: DC Number + line item number, for Delivery Challan records. For movement records, Material Document Number + Movement Type is the likely candidate.

Open Questions / Decisions Required:

- **[OPEN]** Does SAP resend the same DC/PO record on every sync even if unchanged, or only send deltas? This determines whether "no change" detection is needed in addition to duplicate detection.
- **[OPEN]** What identifies an "update" versus a genuinely new record — is it purely the unique key match, or does SAP provide a change timestamp/version we should compare?
- **[OPEN]** Insert logic: confirmed as straightforward — no existing match, create new record.
- **[OPEN]** Update logic: when a match is found, should every field be overwritten with SAP's value (SAP as source of truth), or should certain application-only fields (e.g., internal verification status) be preserved? This needs a field-by-field decision, likely resolved in Section 12.
- **[OPEN]** Conflict handling: if the application has already progressed a DC to a later stage (e.g., QC Acknowledged) and SAP sends an update to that same DC, should the update be applied, rejected, or flagged for review?
- **[OPEN]** Does SAP ever send a "deleted" or "cancelled" status for a DC/PO, and if so, how should the application reflect that on a record that may already be mid-process?

None of these should be assumed — they materially affect the mapping matrix and the processing flow above, so they're flagged here rather than resolved with a guess.

12. Database Impact

SAP Category	Application Table	New Columns Needed	SAP the Source of Truth?
Delivery Challan	DELIVERY_CHALLAN	None — existing fields cover DC No, line item count	Yes, for DC content. Verification status remains application-owned.
Purchase Order	DISPATCH	po_number , po_line_item (new)	Yes
Vendor Master	VENDOR	None — existing fields cover code/name	Yes, but application may extend with local-only fields (e.g., internal notes) — [OPEN] if any exist
Movement Type / Material Document	New table recommended: SAP_MOVEMENT	id , dispatch_id (FK), movement_type , sap_document_no , posting_date , sync_status	Yes
Stock Movement / Storage Location	STORAGE_LOCATION	Possibly sap_storage_location_ref if SAP's location concept doesn't map 1:1 to Rack/Bin	[OPEN] — depends on Section 5's open item
Open PO / Pending Quantity	Not stored directly; computed	—	N/A if computed; if SAP sends it directly, needs a column — [OPEN]

Every new/existing table involved already carries the standard audit columns (created_by , created_at , updated_by , updated_at) from the shared AuditMixin , per the architecture doc's Section 5.3. A sync_status field (e.g., synced , pending , error) is recommended on the new SAP_MOVEMENT table specifically, to support the error/retry flow in Section 10 without needing a separate tracking table.

13. API / Integration Contract (Proposed)

Since the real SAP REST endpoints are not yet shared, the structure below is **Proposed**, not confirmed, and exists to give developers something concrete to build the mock client against.

Aspect	Proposed
Endpoint	<code>GET /dispatched-materials</code> (already stubbed in the backend's <code>SAPRestClient</code>)
Method	GET for extraction, POST for UD Post / SAP 313 (already scaffolded)
Authentication	Bearer token — [OPEN] confirm actual mechanism (API key, OAuth2, certificate)
Request structure	None for GET; JSON body for POST (UD Post / SAP 313 payloads)
Response structure	JSON array of records matching Section 3's data dictionary — [OPEN] exact shape
Status codes	Standard REST (200 success, 4xx client error, 5xx server error) — assumed, not confirmed
Error response	[OPEN] — need SAP's actual error payload format to map into the application's error handling (Section 10)
Validation rules	Applied application-side per Section 7, regardless of what SAP validates on its end
Idempotency	Already designed for on the POST side (<code>Idempotency-Key</code> header, per architecture doc Section 5.2) — [OPEN] whether SAP's endpoints actually support/require this
Retry behavior	Per Section 10's error flow

14. Security

Per the architecture document's existing security section (7.1–7.6), the following applies directly to this module without needing new assumptions:

Authentication/Authorization: SAP REST calls carry credentials server-side only (in the FastAPI backend); the frontend never talks to SAP directly. Exact credential type is **[OPEN]** (Section 13).

Secure communication: TLS enforced for all SAP calls, consistent with the application-wide rule.

Credentials/secrets management: SAP API key/credentials stored as environment variables via the existing secrets pattern (`SAP_BASE_URL` , `SAP_API_KEY` already scaffolded in `app/core/config.py`), never in source control.

Access control: Only the backend's SAP integration adapter has network access to SAP; no other module calls SAP directly.

Audit logging: Every SAP call (success or failure) is logged with a request ID, consistent with the architecture's structured logging requirement — this extends naturally to cover this module's sync jobs.

Sensitive data handling: Vendor and material data from SAP is business data, not personal data, so no special PII handling is anticipated — **[OPEN]** if this assumption is wrong (e.g., if any SAP field contains personal contact information).

15. Logging & Monitoring

Every sync attempt (scheduled job or manual trigger) should log: timestamp, number of records fetched, number successfully processed, number failed, and failure reasons. This builds directly on the `sap_sync_job` already scaffolded in `app/workers/jobs.py`. The dashboard (M2) is the natural place to surface a "last successful sync" timestamp and a pending-error count, consistent with its existing KPI-card design.

16. Reconciliation

Periodic reconciliation (e.g., a daily job comparing SAP's current Open PO / Pending Quantity against what the application has recorded) is recommended to catch drift — cases where a record was missed, silently failed, or updated in SAP after the application's last sync. **[OPEN]**: exact reconciliation frequency and what should happen when a mismatch is found (auto-correct vs. flag for review) needs a decision, likely the same conflict-handling decision flagged in Section 11.

17. Synchronization Strategy

Two approaches are possible; which one applies depends on information not yet confirmed:

Option A — Scheduled polling. The application calls SAP's REST endpoint on a fixed interval (the architecture doc already scaffolds this via APScheduler, currently set to 15 minutes as a placeholder). Simple to implement, consistent with the "no Celery, in-process scheduler" decision already made for this project. Best if SAP data doesn't need to reflect within seconds of a change.

Option B — Event-driven (webhook/push from SAP). SAP notifies the application when new data is available. Lower latency, but requires SAP to support outbound webhooks, which is **[OPEN]** — not confirmed either way.

Recommendation given what's confirmed so far: since the integration is confirmed as REST (client-initiated), Option A (scheduled polling) is the safer default and is already what's scaffolded. This should be revisited if SAP later confirms webhook support and near-real-time data becomes a requirement.

18. Business Rules

Rules already confirmed elsewhere in the project that this module must respect:

- A single DC may contain a maximum of 13 line items; more requires splitting into multiple DCs (existing rule, enforced server-side per the architecture doc).
- Received quantity is compared against expected quantity as Excess or Less; Less triggers the deviation/revert workflow (M7), which this module's Delivery Challan data feeds directly.

- Movement types 101/313/321 correspond to specific, already-documented physical stages (Goods Receipt, Stock Transfer, Quality to Unrestricted) rather than being arbitrary codes — this module should preserve that mapping rather than treating them as opaque values.

19. Open Questions / Decisions Required (Consolidated)

1. Exact field-level schema for each SAP data category (Section 3) — names, types, formats.
2. Whether SAP integration is pull (polling) or supports push/webhook (Section 17).
3. Authentication mechanism for the SAP REST API (Section 13).
4. Exact SAP error response format (Section 13).
5. Whether unresolvable Vendor/Material references should be rejected or auto-created as stubs (Section 7).
6. Whether SAP sends deltas only or full records on every sync (Section 11).
7. Whether SAP provides a change timestamp/version for update detection (Section 11).
8. Field-by-field ownership: which fields SAP always overwrites vs. which the application owns after initial creation (Section 11).
9. Conflict handling when SAP updates a record the application has already progressed past (Section 11).
10. Whether SAP sends deletion/cancellation signals for DC/PO records (Section 11).
11. Whether SAP's Storage Location concept maps directly to the application's Rack/Bin model (Section 5, Section 12).
12. Whether Open PO / Pending Quantity is sent directly or must be computed application-side (Section 5).
13. Maximum retry attempts and backoff intervals for transient errors (Section 10).
14. Reconciliation frequency and mismatch resolution behavior (Section 16).
15. Whether any SAP field contains data requiring special handling beyond standard business-data treatment (Section 14).

This list should be the first thing shared with the SAP team before implementation begins on this module, since nearly every section above has at least one dependency on it.

20. Final End-to-End Summary

Titan SAP is the source of Delivery Challan, Purchase Order, Vendor Master, and stock movement data (movement types 101/313/321), reaching the PPC CBE Tracking Application via a REST integration that the backend already scaffolds behind an adapter pattern. Each incoming record is mapped field-by-field into the application's existing entities (extending a few tables where noted), validated against both structural and business rules, transformed, and persisted with full audit trail and traceability back to its

SAP reference. Errors are split into transient (retried) and permanent (logged and routed for manual review), and the whole flow is designed to be built and tested today against a mock client, so implementation isn't blocked on SAP access.

The module's biggest risk is not technical but informational: fifteen distinct open questions (Section 19) determine the mapping matrix, the duplicate/update logic, and the synchronization strategy. Building ahead of those answers risks having to rework core logic once real answers arrive, particularly the update/conflict logic in Section 11, which should not be implemented until that section is actually resolved.